

Question 1

Model Used

The given problem is solved using the **Constraint Satisfaction Problem (CSP) Model** with the **Backtracking Search Algorithm**. In this model, the doctors are considered as variables, the shifts are treated as the domain, and the scheduling rules are represented as constraints. The Backtracking Search Algorithm systematically assigns shifts to doctors and checks whether all constraints are satisfied. Whenever a constraint is violated, the algorithm backtracks and explores another possible assignment until a valid schedule is obtained.

Problem Representation

Variables

$$D = \{D1, D2, D3\}$$

Domain

$$S = \{Morning, Afternoon, Night\}$$

Constraints

- $D1 \neq \text{Night}$
- $D2 < D3$
- One doctor per shift
- $D3 \neq \text{Morning}$
- Every doctor must be assigned exactly one shift

State Representation

A state is represented as

$$(D1, D2, D3)$$

Example

(Morning, Afternoon, Night)

which represents

D1 → Morning

D2 → Afternoon

D3 → Night

Initial State

$(-, -, -)$

No doctor has been assigned a shift.

Goal State

(Morning, Afternoon, Night)

or any other assignment that satisfies all the constraints.

Constraint Checking

Step State		Status
1	(Morning, -, -)	Valid
2	(Morning, Morning, -)	Invalid
3	(Morning, Afternoon, -)	Valid
4	(Morning, Afternoon, Afternoon)	Invalid
5	(Morning, Afternoon, Night)	Goal Reached

Search Representation

Start

|

(-, -, -)

|

D1 = Morning

|

D2 = Morning X

|

Backtrack

|

D2 = Afternoon

|

D3 = Afternoon X

|

Backtrack

|

D3 = Night ✓

Algorithm

The Backtracking Search Algorithm starts with an empty assignment and selects one doctor at a time. After assigning a shift, it verifies whether all constraints are satisfied. If a constraint is violated, the algorithm returns to the previous state and selects another available shift. This process continues until a complete and valid assignment is obtained.

Final Assignment

Doctor Shift

- D1 Morning
- D2 Afternoon
- D3 Night

Applications

Hospital duty scheduling, employee shift allocation, examination timetable generation, resource allocation, and workforce scheduling.

Result

The hospital scheduling problem was successfully solved using the **Constraint Satisfaction Problem (CSP) Model** and the **Backtracking Search Algorithm**. The final assignment satisfies all the given constraints and produces a valid schedule.

Question 2

Model Used

The given problem is solved using the **Breadth-First Search (BFS) Model**. BFS is an uninformed search algorithm that explores the search space level by level. Since every movement in the grid has the same cost (1), BFS guarantees the shortest path from the Start node to the Goal node. The Manhattan Distance is used only as a heuristic to estimate how close a node is to the goal, while BFS performs the actual search.

Problem Representation

Grid Size

$$5 \times 5$$

Initial State

$$S = (0,0)$$

Goal State

$$G = (4,4)$$

Possible Actions

- Move Up
- Move Down
- Move Left
- Move Right

Movement Cost

$Cost = 1$ for every move

Heuristic Used

The Manhattan Distance is calculated as

$$h(n) = |x_g - x| + |y_g - y|$$

where

- x_g, y_g = Goal position
- x, y = Current position

State Representation

Each state is represented as

$$(x, y)$$

Example

$$(2, 3)$$

indicates that the robot is currently at Row 2 and Column 3 of the grid.

Search Space Representation

$S \rightarrow (0,0)$

|

├— (0,1)

|

├— (1,0)

|

├— (1,1)

|

├— (2,1)

|

├— (2,2)

|

├— (3,2)

|

├— (4,2)

|

├— (4,3)

|

└— G (4,4)

Node Expansion

Step Expanded Node Queue

1	(0,0)	(0,1),(1,0)
---	-------	-------------

Step Expanded Node Queue

2	(0,1)	(1,0),(1,1)
3	(1,0)	(1,1),(2,0)
4	(1,1)	(2,0),(2,1)
5	(2,1)	(2,2),(3,1)
6	(2,2)	(3,2),(2,3)
7	(3,2)	(4,2),(3,3)
8	(4,2)	(4,3)
9	(4,3)	(4,4)
10	(4,4)	Goal Reached

Shortest Path

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

Path Cost

Since every movement costs **1**,

Total Cost = 8

Algorithm

The Breadth-First Search algorithm starts from the initial node and explores all neighboring nodes level by level using a queue. Every newly discovered node is added to the queue only once to avoid revisiting previously explored states. The search continues until the goal node is reached. Since each move has the same cost, BFS always returns the shortest possible path.

Applications

- Robot Navigation
- GPS Route Planning
- Maze Solving
- Path Finding in Games
- Network Routing
- Mobile Robot Navigation

Result

The robot successfully reaches the goal using the **Breadth-First Search (BFS) Model**. The algorithm explores the grid level by level and finds the shortest path with a total movement cost of **8**. The Manhattan Distance heuristic helps estimate the distance to the goal while BFS guarantees the optimal path under uniform movement cost.

Question 3

Model Used

The given problem is solved using the **Online Search Agent** integrated with the **Uniform Cost Search (UCS) Algorithm**. The Online Search Agent is suitable because the rescue robot operates in a partially observable environment where complete information is not available initially. Uniform Cost Search helps the robot choose the least-cost path by considering both the movement cost and the additional risk cost while travelling through the environment.

Problem Representation

Environment

$$E = \{Safe\ Area, Blocked\ Area, Risky\ Area, Survivor\}$$

Initial State

$$S = Robot\ Start\ Position$$

Goal State

$$G = \text{Survivor Location}$$

Actions

$$A = \{\text{Move Up, Move Down, Move Left, Move Right}\}$$

Movement Cost

$$\text{Cost}(\text{Move}) = 1$$

Risk Cost

$$\text{Cost}(\text{Risk}) = 2$$

Total Path Cost

$$f(n) = \text{Movement Cost} + \text{Risk Cost}$$

Agent Representation

The rescue robot continuously senses the surrounding environment, updates its internal knowledge, and selects the safest path toward the survivor. Whenever a blocked path is encountered, the robot immediately changes its route based on the newly available information. This behavior makes the Online Search Agent suitable for dynamic and uncertain environments.

State Representation

Each state is represented as

$$(\text{Current Location}, \text{Total Cost})$$

Example

$$(A, 3)$$

indicates that the robot is currently at location **A** with a cumulative path cost of **3**.

Working Representation

Step	Current Position	Action	Cost
1	Start	Move Forward	1
2	Safe Area	Move Right	2
3	Risk Zone	Additional Risk Cost	4
4	Blocked Path	Choose Alternate Path	4
5	Safe Area	Move Forward	5
6	Survivor Found	Goal Reached	5

Environment Type

The rescue robot works in a **Partially Observable Environment** because it cannot view the complete disaster area at the beginning. It is also **Dynamic**, since obstacles and environmental conditions may change during execution. The environment is **Sequential**, as every decision influences future actions, and it is **Single-Agent**, because only one robot performs the rescue operation.

Intelligent Agent Architecture

Environment

|

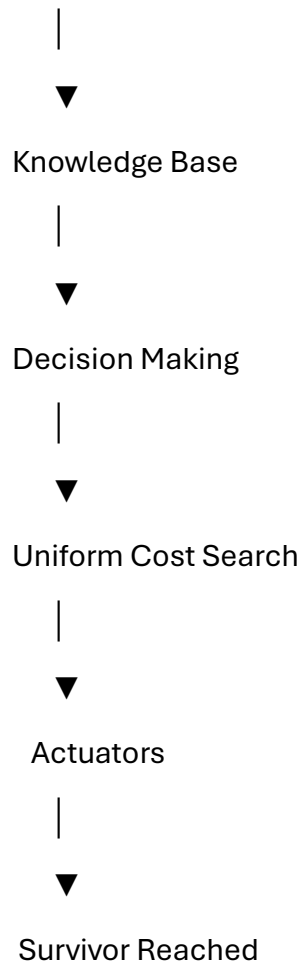


Sensors

|



Perception Module



Rationality

The rescue robot behaves as a **rational agent** because it always selects the action that maximizes the chances of safely reaching the survivor while minimizing the total travel cost. Whenever a blocked path or risky area is detected, the robot updates its decision and chooses a better alternative instead of following the previous path.

Algorithm

The Online Search Agent begins from the starting location and senses the surrounding environment. The discovered information is stored in its knowledge base. Uniform Cost Search is then applied to evaluate all available paths based on their cumulative cost. The robot always expands the least-cost path first. If a blocked path is detected, the robot updates its knowledge and searches for another feasible path. The process continues until the survivor is reached successfully.

Applications

- Disaster Rescue Operations
- Fire Fighting Robots
- Search and Rescue Missions
- Military Surveillance
- Autonomous Delivery Robots
- Space Exploration Rovers
- Warehouse Navigation Systems

Result

The rescue robot successfully reaches the survivor by using the **Online Search Agent Model** together with the **Uniform Cost Search Algorithm**. The robot continuously updates its knowledge about the environment, avoids blocked paths, minimizes the overall travel cost, and makes rational decisions throughout the rescue operation. This approach enables efficient navigation in dynamic and partially observable environments.